DSCI 551 Foundations of Data Management
Database Internals + Application Design

ANALYZING DUCKDB INTERNALS THROUGH A RETAIL ANALYTICS APPLICATION

May 6, 2026

Renad Alqahtani - USC ID 6430884923
ralqahta@usc.edu

1 Introduction and Motivation

This project explores how DuckDB executes analytical queries and how its internal design influences query behavior and performance. DuckDB was selected because it is an in-process analytical database that can be easily integrated within Python environments and is specifically optimized for data analysis workloads.

The goal of this project is to build a simple analytical application and examine how database internals, such as columnar storage and vectorized execution, affect query execution. By connecting internal mechanisms to observable behavior, the project highlights how design decisions in modern databases influence performance in analytical settings.

2 Database System Overview

DuckDB is an in-process analytical database system designed for efficient execution of analytical queries. Unlike traditional database systems that require a separate server, DuckDB runs directly within the host application, making it lightweight and easy to integrate into environments such as Python and Jupyter notebooks.

A key feature of DuckDB is its column-oriented storage model, which allows queries to read only the necessary columns rather than entire rows. This reduces data access and improves performance for analytical workloads. Additionally, DuckDB uses a vectorized execution model, processing data in batches instead of one row at a time, which enables efficient filtering, aggregation, and grouping operations.

DuckDB supports standard SQL, making it accessible to users familiar with relational databases. It can also directly query data from formats such as CSV and Parquet without requiring a separate data loading step, simplifying data analysis workflows.

DuckDB is well suited for data analysis, reporting, and exploratory data processing. It is commonly used in data science workflows, embedded analytics, and scenarios requiring fast, local query execution without the overhead of a database server. Its design makes it particularly effective for read-heavy workloads and large-scale analytical queries.

3 Internal Architecture

This section describes the internal architecture of DuckDB, focusing on how data is stored and how queries are executed. DuckDB is designed specifically for analytical workloads, and its architecture reflects this through a combination of columnar storage and efficient query execution techniques. Understanding these internal components is essential for analyzing how queries behave and why certain operations perform more efficiently than others.

3.1 Storage Model

DuckDB uses a columnar storage format, where data is stored by columns rather than rows. This allows the system to read only the columns required for a query instead of scanning entire rows. This storage format is particularly beneficial for analytical queries, where only a subset of columns is typically accessed. By avoiding unnecessary data reads, the system reduces memory usage and improves query performance.

3.2 Query Execution

DuckDB uses a vectorized execution model, where data is processed in batches instead of row by row. This approach improves performance by reducing overhead and allowing operations to be applied to multiple rows at once. This batch-based processing model minimizes execution cost and allows operations such as filtering and aggregation to be applied efficiently across large datasets. It also enables a consistent execution pipeline where data flows through multiple stages.

3.3 Indexing

DuckDB does not use traditional row-level indexes such as B-trees. Instead, it relies on zone maps (also called min-max indexes), which store the minimum and maximum values for each column chunk. During query execution, DuckDB uses these zone maps to skip chunks that cannot satisfy a filter condition, reducing the amount of data scanned without the overhead of maintaining explicit index structures. This approach is well suited for analytical workloads where range filters and aggregations are common.

3.4 Distribution and Replication

DuckDB is a single-node database system and does not natively support distributed query execution or replication. All data is stored and processed locally within a single process. This design simplifies deployment and eliminates network overhead, but means DuckDB is not suited for distributed environments where data needs to be partitioned across multiple nodes or replicated for fault tolerance.

3.5 Execution Operators

DuckDB processes queries using a series of execution operators that form a pipeline. The most relevant operators observed in this project include:

- **TABLE_SCAN**: Reads data from the table, typically using a sequential scan
- **PROJECTION**: Selects or transforms the required columns
- **HASH_GROUP_BY**: Groups rows and computes aggregate values such as sums

- **ORDER_BY**: Sorts the final query results

These operators appear in execution plans and provide insight into how queries are processed internally. By analyzing these operators, it is possible to understand how different query patterns affect performance and resource usage.

4 Application Design

The application is built using the Superstore dataset, which contains transactional retail data such as sales, profit, category, region, and order date. To better simulate an analytical workload, the dataset was scaled to approximately 200,000 rows.

The application workflow consists of the following steps:

1. Load the dataset into DuckDB
2. Execute analytical queries
3. Use `EXPLAIN ANALYZE` to inspect execution plans
4. Interpret how queries are processed internally

The user interacts with the application through a Jupyter notebook environment. Each notebook cell corresponds to a stage in the pipeline, from loading and scaling the dataset, to executing queries, to inspecting execution plans. Query results are displayed as formatted tables directly within the notebook, and execution plans generated by EXPLAIN ANALYZE are printed inline, allowing the user to observe and interpret DuckDB's internal behavior without leaving the notebook interface.

The application includes four representative queries:

- Aggregation by region
- Filtered aggregation
- Time-based aggregation
- Multiple aggregations

These queries were selected to demonstrate different analytical patterns and how they interact with the database system.

5 Mapping Internals to Application Behavior

The execution plans generated using `EXPLAIN ANALYZE` reveal how DuckDB processes queries through a sequence of operators. By examining these plans, it is possible to directly connect application-level operations to the underlying database internals. Each query follows a similar execution structure, but differences in filtering, transformation, and aggregation influence performance.

The execution pipeline typically begins with a `TABLE_SCAN`, where the required columns are read from storage. Due to DuckDB's columnar storage model, only the columns necessary for a query are accessed, reducing unnecessary data reads. This is followed by a

PROJECTION step, where columns are selected or transformed if needed. For example, in time-based queries, this step is used to derive new values such as extracting the month from a date.

Next, aggregation is performed using the HASH_GROUP_BY operator, which groups rows and computes aggregate values such as sums. This step significantly reduces the size of the data. Finally, an optional ORDER_BY operator is applied to sort the output.

A key example is the filtered aggregation query, which computes total sales by region for a specific category. In this case, DuckDB applies the filter condition during the table scan, reducing the number of rows processed before aggregation. This demonstrates how filtering improves efficiency by limiting the workload early in execution.

Across all queries, several consistent patterns are observed:

- Only required columns are accessed, demonstrating columnar storage
- Sequential scans efficiently process large datasets
- Aggregation reduces data size significantly
- Additional transformations increase computation cost

For example, the basic aggregation query scans the full dataset, while the filtered query reduces rows before aggregation. The time-based query introduces additional computation, and the multi-aggregation query computes multiple values in a single pass. These differences highlight how query structure impacts execution behavior.

Table 1: Summary of Query Behavior and Execution Characteristics

Query	**Columns**	**Operators**	**Observation**
Q1: Sales by Region	Region, Sales	TABLE_SCAN, HASH_GROUP_BY	Full dataset aggregation
Q2: Filtered Sales	Category, Region, Sales	TABLE_SCAN (filter), HASH_GROUP_BY	Filter reduces rows early
Q3: Monthly Sales	Order Date, Sales	TABLE_SCAN, HASH_GROUP_BY	Extra computation for grouping
Q4: Sales and Profit	Category, Sales, Profit	TABLE_SCAN, HASH_GROUP_BY	Multiple aggregates in one pass

6 Comparison with MySQL and MongoDB

DuckDB differs from traditional systems such as MySQL and MongoDB in both architecture and intended use.

MySQL is a row-oriented relational database designed for transactional workloads (OLTP). It supports indexing and efficient point queries, making it suitable for applications with frequent inserts, updates, and lookups. However, it is less efficient for large analytical queries that require scanning substantial portions of data.

MongoDB is a document-oriented NoSQL database designed for flexibility and scalability. It is well suited for semi-structured data and distributed environments. While it provides aggregation capabilities, it is not optimized for complex analytical queries over structured datasets.

DuckDB, in contrast, is designed for analytical workloads (OLAP). Its columnar storage enables reading only required columns, and its vectorized execution model allows efficient batch processing. This makes it highly effective for read-heavy queries and large aggregations within a single-node environment. However, it is not designed for high-concurrency transactional workloads or distributed deployments, limiting its usability in multi-user production environments where simultaneous write operations are required.

Table 2: Comparison of DuckDB, MySQL, and MongoDB

Feature	**DuckDB**	**MySQL**	**MongoDB**
Data Model	Columnar relational	Row-based relational	Document-based
Workload	OLAP analytics	OLTP transactions	Flexible / mixed
Query Language	SQL	SQL	MQL
Scalability	Single-node	Multi-node setup	Horizontal scaling
Schema	Fixed	Fixed	Flexible
Best Use Case	Analytical queries	Transactional systems	Semi-structured data

Overall, DuckDB is optimized for analytical processing, MySQL for transactional workloads, and MongoDB for flexible and scalable applications. The choice depends on the specific workload requirements.

7 Limitations and Lessons Learned

This project demonstrates how DuckDB executes analytical queries and how its internal design influences query behavior. For the purposes of this project and demonstration, a scaled version of the Superstore dataset was used to simulate an analytical workload while keeping the system manageable and easy to analyze. In future work, larger and more complex datasets could be explored to further evaluate performance and scalability.

The current implementation focuses on core analytical functionality within a Jupyter notebook environment. While this setup is effective for experimentation and analysis, the application could be extended into a more complete system with a user interface or dashboard to improve usability and interaction. This extension was not included due to time constraints but represents a natural next step for future development.

One of the main challenges in this project was interpreting the execution plans produced by `EXPLAIN ANALYZE`. These plans include detailed low-level information, and identifying the most relevant components required careful analysis. By focusing on key operators such as table scans, projections, and aggregation, it became possible to clearly connect database internals to query behavior.

Through this project, a deeper understanding of analytical database systems was developed, particularly how columnar storage and vectorized execution improve performance. It also highlighted the importance of aligning database design choices with the intended workload. Overall, this project reinforced how internal database mechanisms directly impact application behavior and efficiency.

8 Project Links

The full implementation of this project, including the Jupyter notebook, dataset setup, and supporting files, is available at the following GitHub repository:

`https://github.com/Renad-A/DSCI-551-Project.git`

References

[1] DuckDB Foundation. Duckdb documentation – architecture and internals, 2024. Accessed: 2026-02-18. URL: `https://duckdb.org/docs/`.

[2] Mark Raasveldt and Hannes Mühleisen. Duckdb: an embeddable analytical database. *Proceedings of the VLDB Endowment*, 12(12):1981–1984, 2019.